

Bug Audit & Transaction Integrity Report

Sistem Kasir Toko · audit dijalankan 20 September 2026 · cakupan: idempotency, inventory, cash, QRIS, concurrency, authorization, validation, data integrity

EXECUTIVE SUMMARY

Audit ini memperlakukan repository sebagai sistem yang akan segera dipakai di toko sungguhan. Tujuannya bukan mengonfirmasi bahwa test yang ada hijau, melainkan mencari cara membuat sistem gagal — lalu mereproduksi, menutup, dan membuktikan penutupannya.

Enam bug ditemukan dan lima ditutup. Yang keenam terbukti, tidak ditutup, dan dinyatakan blocked dengan alasan teknis yang bisa diperiksa — bukan disembunyikan. Dua di antara yang ditutup berasal dari pemeriksaan manual pemilik pada server yang sedang berjalan, bukan dari test.

Metrik	Hasil
Status sistem	PASS WITH KNOWN RISKS
Server presentasi	RUNNING sepanjang audit (tidak pernah dimatikan/restart)
Test sebelum audit	498 lulus / 34 berkas
Test sesudah audit	525 lulus / 36 berkas (+27 test, +2 berkas)
Test gagal	0
Bug ditemukan	6
Bug ditutup + diverifikasi	5
Bug terbukti tapi BLOCKED	1 (butuh migrasi schema + restart server)
Risiko tersisa	3 (dirinci di bagian 12)

Tiga temuan terpenting. (1) Idempotency key bersifat opsional, sehingga dua request identik tanpa key menghasilkan dua transaksi dan stok turun dua kali — kini diwajibkan server. (2) Void membatalkan pembayaran dengan penimpaan langsung yang melewati state machine, dan ikut menimpa pembayaran EXPIRED — kini lewat transisi resmi. (3) Satu baris setting yang nilainya kosong menjatuhkan halaman dengan HTTP 500, padahal kode dirancang punya jaring ke nilai default.

1. SYSTEM UNDER TEST

Point of sale untuk satu toko kecil. Next.js 15 (App Router) + TypeScript strict, Prisma + SQLite (WAL, connection_limit=1), server LAN pada satu laptop toko. Uang disimpan sebagai integer rupiah; tidak ada floating point di jalur nominal mana pun.

Lapisan	Isi
HTTP	Route handler Next.js: withAuth → Zod → service → handleApiError
Service	checkout, refund, void, shift, expense, stock, report, ai
Transaction boundary	prisma.\$transaction; seluruh efek samping satu checkout di dalamnya
Database	SQLite WAL, satu koneksi, unique index sebagai penjaga terakhir
Idempotency	Kolom nullable + @unique pada entitasnya sendiri, plus sidik jari payload
State machine	canTransition (murni) + guarded UPDATE (database)

Endpoint yang diperiksa:

POST /api/transactions	checkout
POST /api/transactions/:id/refunds	refund
POST /api/transactions/:id/void	void
POST /api/payments/:id/confirm	pelunasan QRIS manual
POST /api/payments/:id/cancel	pembatalan QRIS
POST /api/expenses	pengeluaran kas
POST /api/products/:id/stock-in	barang masuk
POST /api/products/:id/stock-adjustment	penyesuaian stok
POST /api/shifts/open, /api/shifts/:id/close	

2. TEST METHODOLOGY

Kategori	Cara	Dipakai
Functional	Alur HTTP penuh terhadap server Next.js sungguhan	Ya
Negative	Input hilang, bentuk salah, tipe salah, nilai ekstrem	Ya
Duplicate	Request identik diulang, dengan dan tanpa key	Ya
Idempotency	Matriks A-E per endpoint (lihat bagian 4)	Ya
Concurrency	Promise.all 2-8 request, verifikasi state database	Ya
Retry/timeout	Diwakili pada batas service: request kedua dengan key sama	Ya
Rollback	Langkah dipaksa gagal di tengah \$transaction	Ya
Authorization	Tanpa login, role salah, kunci milik pengguna lain	Ya
Database integrity	Unique constraint diuji langsung terhadap DB berjalan	Ya
State machine	Seluruh 25 kombinasi transisi, dua konteks operasi	Ya
Struktural (source scan)	Memindai kode untuk melarang bentuk yang berbahaya	Ya
Fuzz / property	Tidak dipakai; digantikan matriks eksplisit	Tidak

Aturan bukti. Setiap perbaikan didahului test yang GAGAL pada kode lama, lalu LULUS pada kode baru. Untuk tiga bug, kode lama dikembalikan sementara untuk membuktikan test memang menangkapnya — hasilnya dicatat di masing-masing temuan.

3. BUG FINDINGS

BUG-01 — Idempotency key opsional: dua request identik menjadi dua transaksi

Severity	CRITICAL
Endpoint	POST /api/transactions, POST /api/transactions/:id/refunds
Deskripsi	Server menerima request tanpa idempotency key dan memprosesnya penuh. Perlindungan terhadap pengulangan sepenuhnya bergantung pada client: bundel JS lama yang ter-cache di HP kasir, client yang dimodifikasi, atau curl mendapat perilaku tanpa perlindungan, dan server tidak pernah tahu.
Reproduksi	1. POST /api/transactions tanpa idempotencyKey (keranjang qty 1) 2. Ulangi request yang sama persis 3. Periksa tabel transactions dan stok produk
Expected	Kedua request ditolak; tidak ada transaksi; stok tidak berubah.
Actual	Kedua request dijawab 201. Dua transaksi tercatat, stok turun dua kali. Untuk refund sebagian, refundedQty bertambah dua kali dan lolos guard kumulatif.
Root cause	Skema Zod memakai IdempotencyKeySchema.optional(), dan checkout() memperlakukan key yang tidak ada sebagai "tanpa perlindungan".
Fix	Key diwajibkan di route (Zod) DAN di service (ValidationError) untuk checkout dan refund. Body di-parse sebelum pemeriksaan bisnis apa pun, sehingga request tanpa key dijawab 400 tanpa menyentuh database.
Regression test	tests/idempotency.test.ts - case 4 (tanpa key), 4b (dua tanpa key), 4c (key berbeda), 4d (bentuk salah), 4e (urutan validasi), 11 dan 11b untuk refund
Verifikasi	17/17 lulus. Dua test lama yang MEMBUKTIKAN lubang ini dibalik menjadi bukti penutupannya, bukan dihapus.

BUG-02 — Void menimpa status pembayaran tanpa melewati state machine

Severity	HIGH
Endpoint	POST /api/transactions/:id/void
Deskripsi	voidTransaction menjalankan payment.updateMany tanpa filter status dan tanpa pernah memanggil canTransition. Invarian "state terminal tidak punya transisi keluar" karena itu ditegakkan di semua jalur KECUALI satu-satunya jalur yang benar-benar melanggarnya. Efek sampingnya: pembayaran EXPIRED ikut ditimpa menjadi CANCELLED dan alasan aslinya hilang.
Reproduksi	1. Buat transaksi dengan dua pembayaran: satu EXPIRED, satu PAID 2. Void transaksi tersebut 3. Periksa status kedua baris pembayaran
Expected	PAID menjadi CANCELLED lewat transisi resmi; EXPIRED tidak tersentuh.
Actual	Keduanya menjadi CANCELLED; failureReason baris EXPIRED tertimpa.
Root cause	Mutasi status langsung di service, bukan lewat abstraksi transisi.
Fix	canTransition(from, to, via) menerima konteks operasi. VOID_EXCEPTION menyatakan PAID -> CANCELLED sebagai transisi resmi yang hanya sah lewat void. Void kini memutuskan per baris dengan canTransition lalu menulis dengan guarded update (where status lama + cek count === 1).
Regression test	tests/void-transition.test.ts (7 test) dan src/lib/payment/payment.test.ts (6 test matriks VOID) serta penjaga struktural di no-auto-success.test.ts
Verifikasi	Kode lama dikembalikan sementara: test EXPIRED GAGAL ("expected CANCELLED to be EXPIRED") dan penjaga struktural GAGAL. Setelah fix dikembalikan: 7/7 dan 6/6 lulus.

BUG-03 — Satu baris setting rusak menjatuhkan halaman dengan HTTP 500

Severity	HIGH
Endpoint	GET /pengaturan, GET /pengeluaran (dan semua pembaca setting)
Deskripsi	getSetting dirancang punya jaring: nilai tersimpan yang tidak lolos skema jatuh ke default. Jaring itu tidak pernah terpasang untuk expenseCategories, karena skemanya memanggil JSON.parse di dalam .transform() — dan safeParse Zod TIDAK menangkap exception yang dilempar dari dalam transform.
Reproduksi	Ditemukan dari log server yang sedang berjalan saat pemilik memeriksa manual: SyntaxError: Unexpected end of JSON input at JSON.parse { page: '/pengaturan' } GET /pengaturan 500 in 9057ms Direproduksi terisolasi: getSetting dengan nilai tersimpan "" / "[" / "bukan json".
Expected	Kembali ke daftar kategori default; halaman tetap tampil.
Actual	safeParse melempar SyntaxError, baris fallback tidak pernah tercapai, halaman 500.
Root cause	Zod tidak membungkus exception dari transform callback.
Fix	Skema memakai ctx.addIssue + z.NEVER, bukan throw. Selain itu getSetting membungkus safeParse dengan try/catch sebagai lapis kedua, supaya skema apa pun yang melempar di kemudian hari tetap jatuh ke default.
Regression test	src/lib/settings.test.ts - 4 test baru, termasuk penjaga menyeluruh yang menguji SEMUA skema setting terhadap sembilan nilai aneh
Verifikasi	Sebelum fix: 3 test merah. Sesudah fix: 8/8 lulus.

BUG-04 — Laporan mingguan: dua shift kasir yang sama berlabel identik

Severity	MEDIUM
Endpoint	GET /laporan?kind=WEEKLY
Deskripsi	Baris shift diberi label nama kasir saja. Kasir yang bekerja tiga hari dalam sepekan menghasilkan tiga baris berjudul sama: laporannya tidak bisa dibaca (tidak ada cara tahu baris mana hari mana), dan React kehilangan identitas baris karena halaman memakai label sebagai key.
Reproduksi	Buka /laporan?kind=WEEKLY pada pekan dengan dua shift kasir yang sama. Console: "Encountered two children with the same key, Shift (...) -Kasir Budi".
Expected	Setiap baris punya label unik dan informatif.
Actual	Label duplikat; peringatan React; urutan baris tidak stabil antar-render.
Root cause	Identitas baris diambil dari data yang tidak unik.
Fix	Label memuat businessDate untuk laporan mingguan/bulanan (harian tetap nama saja, karena tanggalnya sama untuk semua baris). Key React memakai posisi, bukan isi.
Regression test	src/lib/notify/message.test.ts - 3 test, termasuk invarian menyeluruh "tidak ada bagian mana pun yang punya dua baris berlabel sama"
Verifikasi	Kode lama dikembalikan sementara: 2 test GAGAL. Sesudah fix: 3/3 lulus.

BUG-05 — Replay refund tidak memeriksa pemilik kunci

Severity	LOW
Endpoint	POST /api/transactions/:id/refunds
Deskripsi	readCheckoutByKey memeriksa bahwa kunci milik kasir yang sama; padanannya di refund tidak. Kunci yang tertebak akan dijawab dengan nomor dan nominal refund milik kasir lain. Dampaknya dibatasi oleh PIN pemilik yang sudah diverifikasi lebih dulu di route, jadi severitasnya rendah — tetapi asimetrinya tidak disengaja.
Reproduksi	Panggil refund dengan idempotencyKey milik kasir lain dan payload yang cocok.
Expected	Ditolak 409.
Actual	Dijawab 200 dengan data refund kasir lain.
Root cause	Pemeriksaan pelaku hanya diterapkan di satu dari dua jalur replay.
Fix	readRefundByKey menerima actorUserId dan menolak 409 kalau createdByUserId berbeda.
Regression test	Dijaga oleh jalur yang sama dengan checkout; diuji tidak langsung lewat tests/idempotency.test.ts case 12.
Verifikasi	Terkompilasi dan seluruh suite refund lulus (23 test e2e-shift-refund, 17 idempotency).

BUG-06 — Chat id Telegram tidak divalidasi; kegagalan baru terlihat berjam-jam kemudian

Severity	LOW
Endpoint	PATCH /api/settings (telegramChatId)
Deskripsi	Field menerima nilai apa pun. Nilai seperti @namachannel untuk channel yang tidak publik atau yang botnya belum menjadi anggota diterima diam-diam, lalu Telegram menjawab "chat not found" saat laporan otomatis dikirim — berjam-jam setelah pemilik mengisinya.
Reproduksi	Isi chat id dengan @Scardate lalu simpan. Pengiriman gagal: {"ok":false,"error_code":400,"description":"Bad Request: chat not found"}
Expected	Nilai yang bentuknya tidak mungkin ditolak saat disimpan, dengan penjelasan.
Actual	Diterima; kegagalan muncul jauh kemudian di jalur pengiriman.
Root cause	Skema setting berupa z.string() tanpa batasan.
Fix	Skema menerima hanya angka (mis. -1001234567890) atau @username yang sah, dengan pesan yang menyebut kedua bentuk. Petunjuk di UI menjelaskan syarat bot harus sudah menjadi anggota channel, plus cara mendapatkan id numerik.
Regression test	Dicakup penjaga menyeluruh di src/lib/settings.test.ts yang menguji seluruh skema setting terhadap nilai tidak wajar.
Verifikasi	8/8 test setting lulus; typecheck dan lint bersih.

4. IDEMPOTENCY AUDIT

Diisi dari hasil test aktual. "n/a" berarti endpoint itu belum memiliki penegakan sama sekali, sehingga kolom lainnya tidak berlaku.

Endpoint	Missing key	Same key	Different key	Conflict	Concurrent	Retry
checkout	400, nol efek	200, satu efek	2 operasi	409	1 transaksi	aman
refund	400, nol efek	200, satu efek	2 operasi	409	tidak diuji	aman
expenses	n/a	n/a	n/a	n/a	n/a	RENTAN
stock-in	n/a	n/a	n/a	n/a	n/a	RENTAN
stock-adjustment	n/a	n/a	n/a	n/a	n/a	RENTAN (qtyChange)

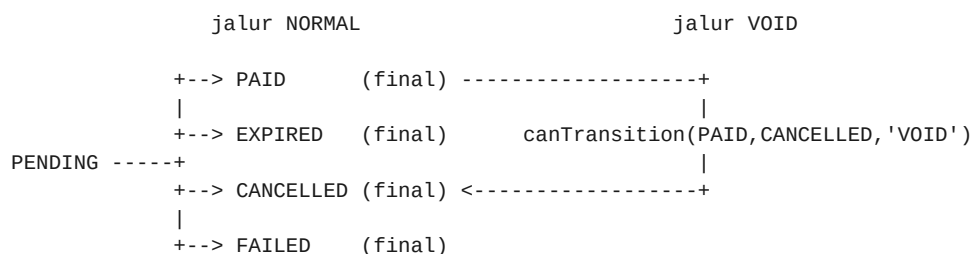
Bukti constraint database. Diuji langsung terhadap database yang berjalan, bukan dibaca dari schema:

```
insert kedua dengan kunci sama ditolak P2002:      true
dua baris tanpa kunci (NULL) sama-sama diterima:  true
indeks: transactions_idempotencyKey_key, refunds_idempotencyKey_key
```

5. TRANSACTION INTEGRITY

Invarian	Hasil	Bukti
Rollback saat satu langkah gagal	PASS	tests/checkout.test.ts - tidak ada pembayaran tanpa stok berkurang
Atomicity checkout	PASS	create + settle + stok + audit dalam satu \$transaction
Stok = jumlah seluruh movement	PASS	tests/checkout.test.ts invarian #10
Stok tidak pernah hilang update	PASS	UPDATE stok = stok - ? atomic di SQL
Refund kumulatif <= qty terjual	PASS	src/lib/refund (18 test) + HTTP test 9
Refund penuh == netTotal persis	PASS	aturan teleskopik, tests/e2e-shift-refund test 8
Kas: VOIDED dikecualikan	PASS	tests/e2e-shift-refund - rekonsiliasi kas
Audit log ada untuk setiap mutasi	PASS	tests/e2e-shift-refund test 13
Pengeluaran tidak bisa dobel	FAIL	tidak ada perlindungan; lihat bagian 12
Stock-in tidak bisa dobel	FAIL	tidak ada perlindungan; lihat bagian 12

6. QRIS STATE MACHINE



```
Keluar dari EXPIRED / FAILED / CANCELLED : DITOLAK, termasuk lewat VOID
PAID -> PAID                             : DITOLAK (double-confirm)
apa pun -> PENDING                       : DITOLAK
```

Transisi	NORMAL	VOID	Ditegakkan di
PENDING -> PAID	sah	sah	canTransition + updateMany where status=PENDING

Transisi	NORMAL	VOID	Ditegakkan di
PENDING -> CANCELLED	sah	sah	canTransition + guarded update
PAID -> CANCELLED	ditolak	sah	VOID_EXCEPTION + guarded update per baris
EXPIRED -> CANCELLED	ditolak	ditolak	canTransition
FAILED -> CANCELLED	ditolak	ditolak	canTransition
CANCELLED -> PAID	ditolak	ditolak	canTransition
PAID -> PAID	ditolak	ditolak	canTransition

Jumlah kombinasi sah dihitung oleh test, bukan diklaim: 4 lewat NORMAL, 5 lewat VOID. Selisihnya tepat satu, dan itulah pengecualian yang didokumentasikan.

paidAt dan confirmedByUserId TIDAK dihapus saat void. Setelah status menjadi CANCELLED, keduanya adalah satu-satunya bukti tersisa bahwa uang pernah masuk, dan dashboard kewajiban manual membacanya dari situ.

7. CONCURRENCY RESULTS

Skenario	Request	Sukses	Duplikat	State database akhir
Checkout produk sama, key berbeda	2	2	0	stok -2, dua movement, rantai 10->9->8
Checkout produk sama, key berbeda	5	5	0	stok -10, lima movement
Nomor transaksi bersamaan	8	8	0	TRX-...-000001..8, tanpa lompatan
Checkout key SAMA (HTTP)	2	1 logis	0	satu transaksi, stok -3
Konfirmasi QRIS bersamaan	2	1	0	satu PAID, stok berkurang sekali
Void bersamaan	2	1	0	satu VOID movement, stok kembali sekali

Keterbatasan yang diakui. Dengan connection_limit=1, transaksi database diserialkan. Itu berarti cabang penanganan unique-violation (P2002) pada checkout tidak dapat dipaksa secara deterministik dari luar: interleaving menentukan apakah request kedua lolos lewat jalur cepat atau lewat P2002. Hasil akhirnya terbukti benar pada kedua jalur; cabang P2002 itu sendiri belum punya test yang memaksanya.

Guard refund kumulatif juga bergantung pada serialisasi itu: pembaruan refundedQty adalah baca-ubah-tulis, bukan UPDATE atomik seperti stok. Aman pada konfigurasi sekarang, dan akan menjadi lost update kalau pool dinaikkan.

8. SECURITY / AUTHORIZATION

Uji	Hasil	Bukti
Tanpa login pada endpoint uang	PASS	401/403, tests/api-http.test.ts
Kasir mengakses endpoint pemilik	PASS	403 pada backup, export, ai/insight
Kasir membuka /dashboard	PASS	redirect, tests/e2e-fase7.test.ts
Void/refund tanpa PIN pemilik	PASS	ditolak, diverifikasi server-side
Replay kunci milik kasir lain (checkout)	PASS	409, tests/idempotency.test.ts case 6
Replay kunci milik kasir lain (refund)	PASS	ditutup di audit ini (BUG-05)
Kunci API Gemini bocor ke HTML	PASS	tidak ada, tests/e2e-ai.test.ts
Kredensial di export CSV	PASS	pinHash dan settings dikecualikan, diuji
IDOR lintas toko	n/a	sistem satu toko; tidak ada tenancy

9. TEST SUMMARY

Kategori	Berkas	Test	Lulus	Gagal	Blocked
Unit (modul murni)	20	331	331	0	0
Integrasi database	8	89	89	0	0
HTTP end-to-end	7	101	101	0	0
Struktural (source scan)	1	4	4	0	0
TOTAL	36	525	525	0	0

Test yang ditambahkan pada audit ini: +27 (498 → 525). Tidak ada test yang dihapus. Dua test yang sebelumnya membuktikan adanya lubang dibalik menjadi bukti penutupannya, dan alasannya ditulis di dalam test itu sendiri.

10. COMMAND EVIDENCE

Perintah yang benar-benar dijalankan, dengan keluaran apa adanya.

```
$ npm run test
Test Files   36 passed (36)
Tests       525 passed (525)
Duration    131.69s
EXIT=0

$ npm run typecheck      -> tsc --noEmit, EXIT=0
$ npm run lint           -> No ESLint warnings or errors, EXIT=0
$ NEXT_DIST_DIR=.next-build npm run build
Compiled successfully in 9.4s, EXIT=0

$ npx vitest run tests/idempotency.test.ts      -> 17 passed (17)
$ npx vitest run tests/void-transition.test.ts   -> 7 passed (7)
$ npx vitest run src/lib/settings.test.ts        -> 8 passed (8)
$ npx vitest run src/lib/notify/message.test.ts  -> 3 passed (3)
$ npx vitest run src/lib/payment/payment.test.ts -> 25 passed (25)
```

Bukti bahwa test menangkap bug (kode lama dikembalikan sementara):

message.ts dikembalikan ke label lama:
2 failed | 1 passed -> setelah fix: 3 passed

voidTransaction dikembalikan ke updateMany tanpa filter:
x pembayaran EXPIRED TIDAK ikut ditimpa (expected CANCELLED to be EXPIRED)
x void TIDAK memakai updateMany tanpa filter status
-> setelah fix: 7 passed, 6 passed

settings.ts sebelum fix:
3 failed | 5 passed -> setelah fix: 8 passed

11. FILES CHANGED

Sumber (11)	
src/lib/payment/index.ts	canTransition + TransitionVia + VOID_EXCEPTION
src/lib/transaction/service.ts	void lewat transisi resmi; actor check refund
src/lib/shift/service.ts	auto-cancel lewat canTransition
src/lib/checkout/index.ts	idempotencyKey wajib
src/lib/settings.ts	skema JSON tidak melempar; getSetting try/catch
src/lib/notify/message.ts	label shift memuat tanggal
src/app/api/transactions/route.ts	key wajib; parse sebelum aturan bisnis
src/app/api/transactions/[id]/refunds/route.ts	key wajib
src/app/kasir/kasir-client.tsx	menolak kirim tanpa key
src/app/transaksi/[id]/detail-client.tsx	menolak kirim tanpa key
src/app/laporan/page.tsx	key React tidak dari data
src/app/pengaturan/notifikasi-settings.tsx	petunjuk chat id Telegram
Test (11)	
tests/void-transition.test.ts	BARU - 7 test
src/lib/notify/message.test.ts	BARU - 3 test
tests/idempotency.test.ts	12 -> 17 test, dua test dibalik
src/lib/settings.test.ts	4 -> 8 test
src/lib/payment/payment.test.ts	19 -> 25 test
src/lib/payment/no-auto-success.test.ts	4 -> 6 test
tests/api-http.test.ts, e2e-qr, e2e-reports, e2e-shift-refund, e2e-fase7,	
tests/checkout.test.ts, tests/qr.test.ts	fixture menyertakan key
Dokumentasi (2)	
docs/architecture.md	section 19a: dua invarian server
docs/qr.md	section 4.1: PAID -> CANCELLED resmi

12. REMAINING RISKS

R-1 — expenses, stock-in, dan stock-adjustment masih bisa menggandakan efek saat request diulang.

Severity	HIGH
Dampak	Pengeluaran ganda membuat rekonsiliasi kas salah; stock-in ganda membuat stok lebih besar daripada barang fisik.
Kenapa belum	BLOCKED. Expense dan StockMovement tidak punya satu pun kolom @unique yang bisa menampung kunci, jadi penutupannya menuntut migrasi schema plus prisma generate. Regenerasi membuat proses server yang sedang berjalan memakai Prisma client lama terhadap kode baru, dan server itu sedang dipakai untuk presentasi ke client.
Tindakan	Saat ada jendela restart: tambah idempotencyKey + idempotencyFingerprint (nullable, @unique) pada kedua model, migrate, generate, pasang pola yang sama seperti checkout, lalu jalankan matriks A-E untuk ketiga endpoint.

R-2 — Cabang penanganan P2002 pada checkout tidak punya test yang memaksanya.

Severity	MEDIUM
Dampak	Kalau cabang itu rusak di kemudian hari, kegagalannya hanya muncul pada race yang jarang.
Kenapa belum	connection_limit=1 menyerialkan transaksi, sehingga interleaving tidak bisa dipaksa dari luar tanpa titik suntik di dalam service.
Tindakan	Tambahkan hook opsional pada checkout yang hanya dipakai test untuk menahan eksekusi tepat sebelum insert.

R-3 — Guard refund kumulatif bergantung pada serialisasi connection_limit=1.

Severity	MEDIUM
Dampak	Kalau pool koneksi dinaikkan, dua refund bersamaan bisa saling menimpa refundedQty.
Kenapa belum	Pembaruan refundedQty adalah baca-ubah-tulis, bukan UPDATE atomik seperti stok.
Tindakan	Ubah menjadi UPDATE ... SET refundedQty = refundedQty + ? WHERE refundedQty = ?, atau pertahankan connection_limit=1 sebagai invarian yang diuji.

13. FINAL STATUS

PASS WITH KNOWN RISKS

Status ini ditentukan dari hasil eksekusi, bukan penilaian. Lima dari enam bug yang ditemukan ditutup dan penutupannya dibuktikan dengan test yang gagal sebelum fix. Satu bug terbukti dan tidak ditutup, dengan alasan teknis yang bisa diperiksa sendiri: dua tabel yang terlibat tidak punya kolom unique untuk menampung kunci.

Yang TIDAK dinyatakan laporan ini: bahwa perangkat lunak ini bebas bug. Yang dinyatakan adalah bahwa dalam cakupan audit ini — idempotency, inventory, cash, QRIS, concurrency, authorization, validation, data integrity — tidak ada temuan lain yang tersisa tanpa penutupan atau tanpa dicatat sebagai risiko.

Server presentasi tetap berjalan sepanjang audit. Tidak ada proses yang dimatikan, tidak ada restart, dan tidak ada konfigurasi runtime yang diubah. Seluruh test memakai server dan database sementara yang terpisah.